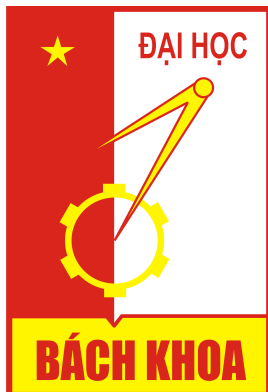


ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO

PROJECT I

ĐỀ TÀI: WEBSITE CHƠI CỜ CARO CÓ KHẢ NĂNG SUY LUẬN

GVHD: TRẦN ĐÌNH KIẾN GIANG
SVTH: NGUYỄN HOÀNG LONG
MSSV: 20235366

Hà Nội- 2025

Mục lục

1	Giới thiệu bài toán	2
1.1	Yêu cầu bài toán	2
1.2	Kế hoạch xây dựng dự án	2
2	Phân tích, thiết kế các chức năng	4
2.1	Đăng nhập, Đăng ký, Đăng xuất	4
2.1.1	Đăng ký (Register)	4
2.1.2	Đăng nhập (Login)	4
2.1.3	Đăng xuất (Logout)	5
2.2	Chế độ chơi PvP, PvF	5
2.2.1	Chế độ PvP	5
2.2.2	Chế độ PvF	6
2.3	Chế độ PvE	7
2.4	Lịch sử trận đấu	9
2.4.1	Xem lại danh sách các trận đấu	9
2.4.2	Hiện thị mức độ biến động elo	9
2.4.3	Xem lại Replay của từng trận đấu	9
2.4.4	Tìm kiếm trận đấu theo username, ngày thi đấu	9
2.5	Thống kê	10
2.6	Chat	10
2.7	Profile Picture	11
2.7.1	URL	11
2.7.2	UPLOAD	11
3	Thiết kế chương trình	12
3.1	Thiết kế kiến trúc	12
3.2	Thiết kế CSDL	12
4	Kết quả chương trình minh họa	14
5	Công cụ sử dụng	19
6	Kết luận, hướng phát triển	21
6.1	Kết luận	21
6.2	Hướng phát triển	21
7	Tài liệu tham khảo	22

Chương 1

Giới thiệu bài toán

1.1 Yêu cầu bài toán

1. **(UI)**: trực quan, dễ sử dụng.
2. **(Đăng nhập/ Đăng ký)**: user có thể đăng nhập, đăng ký tài khoản thuận tiện, và password được mã hóa để tăng mức độ bảo mật.
3. **(CSDL)**: lưu trữ, quản lí DB user, trận đấu trên DBMS MySQL.
4. **(khả năng kết nối thời gian thực)**: chạy mượt mà ở cả 3 chế độ PvP, PvE, PvF.
5. **(chế độ PvP)**: tự động ghép cặp (match matching) những user có cùng mức elo.
6. **(chế độ PvE)**: ứng dụng thuật toán Minimax, Alpha-Beta pruning để xây dựng.
7. **(chế độ PvF)**: có khả năng tạo ROOM để bạn bè chơi với nhau.
8. **(lịch sử trận đấu)**:
 - (a) có thể xem lại danh sách các trận đấu của từng user.
 - (b) hiện thị mức độ biến động elo sau mỗi trận đấu.
 - (c) có thể xem lại replay của từng trận đấu.
 - (d) có thể thực hiện tìm kiếm trận đấu theo username, ngày thi đấu.
9. **(thống kê)**: có thống kê elo, tỉ lệ thắng/thua/hòa
10. **(chat)**: 2 user có thể chat trong thời gian trận đấu diễn ra, và messages sẽ được lưu lại trong DB.
11. **(profile)**: có thể thay đổi profile picture bằng URL, UPLOAD.
12. **(Đăng xuất)**: thoát về trang Đăng nhập

1.2 Kế hoạch xây dựng dự án

Kế hoạch thực hiện

Tuần	Công việc	Trạng thái mong muốn
1	Phân tích, thiết kế hệ thống	Có được sơ đồ kiến trúc, luồng hoạt động, các bảng CSDL của hệ thống
2	Thiết kế, dựng khung giao diện (frontend)	Có giao diện cơ bản chạy được
3	Xây dựng backend, database	Có server kết nối với MySQL ổn định
4	Hoàn thiện chế độ PvP, PvF	2 user có thể chơi realtime
5	Hoàn thiện chế độ chơi PvE bằng cách tích hợp AI, sử dụng 2 thuật toán Minimax và Alpha-Beta Pruning	Có thể chơi tốt chế độ PvE
6	Thêm các chức năng: ○ Đăng ký, Đăng nhập, Đăng xuất ○ Lịch sử trận đấu ((a)-(d) ở mục 1.1.8.) ○ Chat trong lúc chơi giữa 2 user ○ thống kê ○ profile picture	Tất cả các chức năng chính hoạt động
7	Kiểm thử, tối ưu, fix bug	Game hoạt động tốt, ổn định
8	Hoàn thành báo cáo, demo chương trình	Đồ án hoàn chỉnh

Chương 2

Phân tích, thiết kế các chức năng

2.1 Đăng nhập, Đăng ký, Đăng xuất

Hệ thống sử dụng mô hình **CLIENT-SERVER**, giao tiếp thông qua: **RESTful API** (cho dữ liệu) và **socket.io** (cho trạng thái online).

2.1.1 Đăng ký (Register)

Logic Hoạt Động (Business Logic).

- CLIENT gửi `username`, `email`, `password`.
- SERVER kiểm tra xem `username` đã tồn tại chưa (`SELECT * FROM users WHERE username=?`).
- Nếu rồi, thì sẽ thông báo là `username` đã tồn tại và không cho tạo tài khoản.
- Nếu chưa, sử dụng thư viện **bcryptjs** để hash mật khẩu. Sau đó, lưu thông tin vào DB (`INSERT INTO users`).

Abstraction.

- Sử dụng mô hình **MVC**: `authRoutes` (route) $\Rightarrow$ `authController` (controller) $\Rightarrow$ `db` (model).
- Bảo mật: Mật khẩu không bao giờ được lưu dưới dạng văn bản thuần (plain text).

2.1.2 Đăng nhập (Login)

Logic Hoạt Động (Business Logic).

- CLIENT gửi `username`, `password`.
- SERVER tìm thông tin trong bảng `users` trong DB.
- Sử dụng **bcryptjs** để so sánh mật khẩu gửi lên với mật khẩu đã băm trong DB.
- Nếu khớp, sử dụng **jsonwebtoken (JWT)** để tạo một token mã hóa chứa thông tin user (`id`, `username` là đủ, không cần `password`).
- SERVER trả về token + thông tin user.
- CLIENT lưu token vào `localStorage` để duy trì phiên đăng nhập.

Abstraction.

- **(Stateless Authentication)** SERVER không lưu session, chỉ xác thực thông qua token mà CLIENT gửi lên ở header (`Authorization: Bearer <token>`).

2.1.3 Đăng xuất (Logout)

Logic Hoạt Động (Business Logic).

- Do sử dụng **JWT** (Stateless), nên SERVER không cần có API logout.
- CLIENT: xóa token và user ra khỏi `localStorage`.
- Nếu đang ở trong chế độ PvP, PvF, thì sẽ gửi sự kiện `socket.emit('leave game')` để báo cho SERVER biết user đã thoát ra, giúp tránh được trạng thái kẹt trong ROOM.
- Chuyển hướng (navigate) về trang login.

2.2 Chế độ chơi PvP, PvF

2.2.1 Chế độ PvP

Logic Hoạt Động (Business Logic).

- **Cơ chế hàng đợi (Queueing Mechanism).**

- SERVER duy trì một mảng global `pvpQueue=[]` trong bộ nhớ RAM.
- Khi user A chọn PvP, CLIENT gửi sự kiện `join_pvp`.
- SERVER kiểm tra:
 - Nếu `pvpQueue` trống: thêm user A vào hàng đợi, và trạng thái là Waiting.
 - Nếu `pvpQueue` không trống: lấy user B ra, rồi tạo một `roomId` ngẫu nhiên.
- SERVER ghép user A và user B vào ROOM này, phân định xem ai đi trước (X), ai đi sau (O).
- SERVER gửi sự kiện `game_start` cho cả 2 user, để bắt đầu trận đấu.

- **Đồng bộ nước đi (Move Synchronisation).**

- SERVER đóng vai trò là replay server (máy chủ chuyển tiếp).
- Khi user A đánh một nước (CLIENT gửi `move`), SERVER không tính toán lại bàn cờ, mà chuyển tiếp ngay nước đi đó đến user B (SERVER gửi `receive move`).
- Logic kiểm tra thắng/thua (`checkWin`) được thực hiện tại CLIENT, để giảm tải cho SERVER.

- **Kết thúc, tính toán ELO**

- Khi có user thắng, CLIENT gửi `game_over`.
- SERVER gọi `eloCalculator` để tính ELO mới, dựa trên ELO cũ của 2 user. Sau đó cập nhật DB.
- Thông báo kết quả đến 2 user.

Abstraction.

- **socket.io ROOM.** hệ thống tận dụng khái niệm ROOM của socket.io để cô lập luồng giao tiếp. Mỗi đấu là một ROOM riêng biệt. Do đó, nếu CLIENT gửi messages, thì sẽ chỉ gửi vào ROOM này, không ảnh hưởng đến các trận đấu khác.
- **Memory State.** trạng thái của trận đấu đang diễn ra không được lưu trong DB, mà được lưu trong biến `activeGames` trên RAM SERVER. Chỉ khi trận đấu kết thúc, thì mới lưu vào DB.

DS và DB.

- **DS. (In-memory Object** được lưu tại SERVER): đối tượng `activeGames` lưu trạng thái tạm thời của trận đấu.

```
1 activeGames[roomId] = {
2   id: roomId,
3   players: [socketId1, socketId2],
4   user1: { id, username, elo }, // Player X
5   user2: { id, username, elo }, // Player O
6   type: 'pvp'
7 }
```

- **DB (MySQL).** Khi kết thúc trận đấu (sự kiện `game_over`), DL được `INSERT` vào bảng `games` và `UPDATE` vào bảng `users`.
 - bảng `users`: `UPDATE` các thuộc tính `elo`, `wins`, `losses` cho cả 2 user.
 - bảng `games`: `INSERT` vào bảng một dòng mới, gồm toàn bộ các thuộc tính `id`, `player1_id`, `player2_id`, `winner_id`, `game_type`, `start_time`, `end_time`, `p1_elo_change`, `p2_elo_change`.

2.2.2 Chế độ PvF

Logic Hoạt Động (Business Logic).

- Tạo Phòng (Host).
 - user A vào chế độ chơi PvF và ấn TẠO PHÒNG, SERVER sẽ tự tạo ngẫu nhiên một ROOM id $\Rightarrow$ SERVER gửi sự kiện `join_pvf`.
 - SERVER tạo một entry mới trong `activeGames` với trạng thái "Waiting for opponent".
 - user A được đưa vào phòng chờ.
- Vào Phòng (Join).
 - user B nhập mã ROOM do user A cung cấp $\Rightarrow$ CLIENT gửi sự kiện `join_pvf`
 - SERVER kiểm tra: liệu ROOM có tồn tại không? trong ROOM đang có bao nhiêu user?
 - Nếu hợp lệ: ghép user B vào ROOM, gán user A là X, user B là O.
 - SERVER gửi sự kiện `start_game`.
- Xử lý ngoại lệ.
 - Nếu user A thoát ROOM khi ROOM chưa có user nào khác $\Rightarrow$ Hủy ROOM.

- Nếu user B nhập sai ROOM id $\implies$ SERVER trả về lỗi `room_not_found`.
- Nếu user B đang muốn vào ROOM, mà ROOM đã đủ 2 user rồi $\implies$ SERVER trả về lỗi `room_full`.

Abstraction.

- **Room identification.** khác với chế độ PvP (ROOM id ngẫu nhiên, ẩn danh), chế độ PvF coi `roomId` là một **khóa công khai (public key)** để định danh trận đấu thuộc chế độ PvF, cho phép user chia sẻ khóa này để kết nối.
- **Reusability.** chế độ PvF tái sử dụng toàn bộ của `move`, `chat`, `game_over` của chế độ PvP. Điểm khác biệt duy nhất nằm ở bước **Handshake** (bắt tay/ kết nối) ban đầu. Sau bước ghép trận, PvF chính là PvP, tất cả các logic xử lý sau đó, cả trong trận đấu, và sau trận đấu đều giống hệt nhau.

DS và DB.

- **DS. (In-memory Object** được lưu tại SERVER) giống hệt như chế độ PvP, chỉ khác biệt ở biến `type`.

```

1 activeGames[roomId] = {
2     ...
3     type: 'pvf', //Danh dau la choi voi ban.
4 }
```

- **DB (MySQL).** Giống hệt như chế độ PvP.

2.3 Chế độ PvE

- **Kiến trúc và Abstraction.**

- **Class Design.**

- Class chính: `CaroAI` trong file `aiEngine.js`.
- Đây là một Helper Class độc lập, không dính dáng đến DB, socket, nhận input là `board` (mảng 2 chiều) và trả về output là `bestMove` (tọa độ r,c).

- **Phương pháp tích hợp vào chương trình.**

- CLIENT gửi nước đi của user
- SERVER cập nhật bàn cờ ảo trên RAM
- SERVER gọi `caroAI.findBestMove(board)`.
- SERVER nhận kết quả từ AI $\rightarrow$ gửi lại `bestMove` cho CLIENT (AI Move).

- **Thuật toán.**

- **Mimimax (Tư duy chiến thuật).**

- **Nguyên lý.**

- **MAX (AI).** Luôn chọn nước đi có điểm số cao nhất (có lợi cho AI).
- **MIN (user).** AI luôn giả định rằng user đi nước đi hay nhất (làm cho điểm số của AI thấp nhất).

- **Độ sâu (Depth).** Được gán `depth=3`.

- **Alpha-Beta Pruning (Tối ưu hiệu năng).**
 - **Alpha.** Giá trị lớn nhất mà MAX (AI) đã tìm thấy cho đến hiện tại.
 - **Beta.** Giá trị lớn nhất mà MIN (user) đã tìm thấy (TUỖNG ĐƯƠNG giá trị thấp nhất đối với MAX (AI)).
 - **Pruning.** Khi đang tính toán một nhánh, nếu MAX (AI thấy nhánh này chắc chắn tồi hơn một nhánh đã tìm thấy trước đó ($\beta \leq \alpha$), nó sẽ **cắt bỏ (break)** ngay lập tức, mà không tính tiếp các nước con của nhánh đó nữa.
- **Hàm Ước Lượng (Heuristic Function).** chính là hàm `evaluate` trong file `aiEngine.js`.
 - **Cách hoạt động.** Quét cả 8 hướng.
 - **Hệ số điểm (Weights).** Dựa trên biến `SCORES` trong file `aiEngine.js`.
 - 5 ô liên tiếp (`FIVE`): 100,000 điểm (THẮNG TUYỆT ĐỐI).
 - 4 ô thoáng 2 đầu (`OPEN_FOUR`): 10,000 điểm (SẮP THẮNG).
 - 4 ô bị chặn 1 đầu (`CLOSED_FOUR`), 3 ô thoáng (`OPEN_THREE`), ... được gán số điểm giảm dần.
 - **Chiến thuật Tấn công, Phòng thủ.**
 - **Điểm tấn công:** `score+=this.countPatterns(board, piece)`.
 - **Điểm phòng thủ:** `score-=1.2*this.countPatterns(board, opponent)`
 - **Logic.** Nhân 1.2 lần số điểm cho user $\implies$ Ưu tiên chơi an toàn hơn. AI ưu tiên chặn (phòng thủ) hơn là tấn công.
- **Tối ưu hóa không gian tìm kiếm** được thực hiện bởi hàm `getPotentialMoves`. Thay vì duyệt toàn bộ $15 \times 15 = 225$ ô ở mỗi bước, hàm `getPotentialMoves` sẽ chỉ lọc ra những ô đáng đi (những ô neighbour, nằm lân cận với những ô đã đánh).
- **DB.** chế độ PvE hiện tại đang không lưu vào DB.
- **Quy trình (Execution).**
 - Nhận input bàn cờ hiện tại (15×15).
 - Khởi tạo hàm `minimax(board, depth=3, -Infinity, +Infinity, true)`.
 - Đệ quy:
 - Lấy danh sách các nước đi khả thi bằng hàm `getPotentialMoves`.
 - Thử đi từng nước đi giả định.
 - Gọi đệ quy xuống tầng sâu hơn `depth-1`.
 - Sử dụng Alpha-Beta Pruning để loại bỏ nhánh thừa.
 - Trường hợp cơ sở (Base Case), khi `depth===0`:
 - Dừng đệ quy.
 - Gọi hàm `evaluate()` để chấm điểm thế cờ tại thời điểm đó.
 - Trả điểm số ngược các tầng trên.
 - Tìm ra, sau đó trả về nước đi có điểm số cao nhất (`bestMove`) cho SERVER để đánh O.

2.4 Lịch sử trận đấu

2.4.1 Xem lại danh sách các trận đấu

Logic Hoạt Động (Business Logic). CLIENT (History Page) $\Rightarrow$ API (`GET /games/History`) $\Rightarrow$ Controller $\Rightarrow$ DB.

2.4.2 Hiện thị mức độ biến động elo

Logic Hoạt Động (Business Logic). Khi trận đấu kết thúc (SERVER gửi sự kiện `game_over`, thì CLIENT tính toán ELO mới cho cả 2 user, rồi tính toán chênh lệch giữa ELO cũ, mới cho cả 2 user. Cuối cùng, gửi cho SERVER để lưu vào 2 thuộc tính `p1_elo_change`, `p2_elo_change`.

2.4.3 Xem lại Replay của từng trận đấu

Logic Hoạt Động (Business Logic).

- **Lưu trữ.** Khi trận đấu kết thúc, mảng `moves` (chứa các object `r,c,piece`) được `JSON.stringify` và lưu vào cột thuộc tính `x_coord`, `y_coord` trong bảng `game_moves` trong DB.
- **Tải dữ liệu.** CLIENT vào trang `/replay/:id`, gọi API lấy chuỗi JSON này về, và thực hiện `JSON.parse` để ra mảng gốc.
- **Mô phỏng (Simulation).**
 - CLIENT khởi tạo bàn cờ rỗng.
 - Có 2 nút điều hướng, là: `next`, `prev`.
 - Biến `currentStep`, có giá trị thuộc `[0, moves.length() ]`.
 - Khi bấm `Next`: lấy nước đi tại `moves[currentStep]`, rồi điền vào bàn cờ ảo, rồi tăng `currentStep`.
 - Khi bấm `Prev`: lấy nước đi tại `moves[currentStep-1]`, rồi xóa khỏi bàn cờ (set bằng `NULL`), rồi giảm `currentStep`.

Abstraction. State Reconstruction (Tái cấu trúc trạng thái). Thay vì lưu ảnh chụp màn hình của từng bước (tồn dung lượng), hệ thống chỉ lưu "nhật ký thao tác" (Event Log) và dùng thuật toán để diễn lại (Re-enact) trạng thái đó.

2.4.4 Tìm kiếm trận đấu theo username, ngày thi đấu

Logic Hoạt Động (Business Logic). Sử dụng **Query Builder**.

- CLIENT gửi parameters: `?opponent=abc&date=mm/dd/yyyy`.
- SERVER sử dụng kỹ thuật **SQL Dynamic Construction** trong file `gameController.js`. Việc xây dựng câu truy vấn động, giúp linh hoạt kết hợp nhiều điều kiện lọc, mà không cần viết quá nhiều API riêng lẻ.

```
1 let query="SELECT ... FROM games ... WHERE (p1=me OR p2=me)";
2 if (opponent) query+=" AND (u1.name LIKE ? OR u2.name LIKE ?)";
3 if (date) query+=" AND DATE(end_time)=?";
```

2.5 Thống kê

Logic Hoạt Động (Business Logic).

- **Tính toán ELO (backend):**
 - Được kích hoạt ngay khi sự kiện `game_over` diễn ra trong `index.js`.
 - SERVER gọi hàm `calculateNewElo` từ file `eloCalculator.js`.
- **Cập nhật dữ liệu (backend).** Sau khi tính toán, SERVER thực hiện 1 TRANSACTION cập nhật đồng thời vào DB: tăng số trận thắng/thua/hòa, cập nhật ELO mới.
- **Hiển thị (frontend).**
 - Tại file `Dashboard.js`, CLIENT gọi API `/auth/me` để lấy dữ liệu thô.
 - Logic tính win rate được xử lý tại CLIENT để giảm tải cho SERVER.

Abstraction.

- **Seperation of Concerns.** Logic tính toán công thức toán học `eloCalculator.js` được tách biệt hoàn toàn khỏi logic xử lý luồng game (`index.js`) và logic lưu trữ (`db.js`).

DB. Bảng `users` lưu trữ các thuộc tính dưới dạng cộng dồn (aggregated):

- `elo` (INT): điểm xếp hạng (mặc định ban đầu: 1,000).
- `wins`, `losses`, `draws` (INT): số lượng trận thắng/thua/hòa.
- `total_matches`: tổng số lượng trận đã đấu.

2.6 Chat

Logic Hoạt Động (Business Logic).

- sử dụng cơ chế gửi/nhận của **socket.io**.
 - **Gửi.** user A nhập tin nhắn $\Rightarrow$ CLIENT emit sự kiện `send_chat` (kèm theo `content` và `roomId` luôn).
 - **Nhận, Chuyển tiếp.** SERVER nhận tin nhắn $\Rightarrow$ gắn thêm `username` của người gửi $\Rightarrow$ emit sự kiện `receive_chat` đến chỉ 2 user trong ROOM đó (`io.to(roomId).emit(...)`).
- Auto Scroll. CLIENT sử dụng `useRef`, `scrollIntoView` để khung chat luôn cuộn xuống tin nhắn mới nhất.

Abstraction. Stateless/Ephemeral (tính tạm thời).

- Hệ thống thiết kế chat theo dạng **không lưu trữ (Transient)**. Tin nhắn chỉ tồn tại trong bộ nhớ RAM (biến state của REACT) trong suốt quá trình trận đấu diễn ra.
- Khi RELOAD/thoát ROOM, tin nhắn sẽ biến mất.

DB. được lưu trữ ngay lập tức (realtime) vào bảng `messages` trong DB, với tất cả các thuộc tính `id`, `game_id`, `user_id`, `content`, `create_at` đều được lưu lại.

2.7 Profile Picture

2.7.1 URL

Logic Hoạt Động (Business Logic).

- CLIENT gửi một string (link URL ảnh) lên API
- SERVER lưu string này vào DB.

Abstraction. Metadata (Đường dẫn): Chỉ lưu string đường dẫn (Reference) vào DB.

DB. được lưu trữ ở bảng `users`, thuộc tính `avatar` (VARCHAR). frontend sẽ tự xử lý logic hiển thị dựa trên string này.

2.7.2 UPLOAD

Logic Hoạt Động (Business Logic).

- **CLIENT.** sử dụng `FormData` để đóng gói file binary, rồi gửi request `multipart/form-data`.
- **Middleware (Multer).**
 - SERVER sử dụng thư viện **multer** để chặn request lại, trước khi nó vào controller.
 - Tự động lưu file vào folder `uploads/` trên ổ cứng.
 - Đổi tên file theo timestamp (`Date.now()+name`) để tránh trùng lặp.
- **Serving Static Files.** SERVER sẽ cấu hình `app.use('/uploads', ...)` để biến folder này thành một public hosting, cho phép truy cập ảnh thông qua đường dẫn `http://localhost:5000/uploads/ten-anh.jpg`.
- **Controller.** nhận thông tin file đã lưu. Tạo đường dẫn URL, lưu đường dẫn này vào DB.

Abstraction. File ảnh thật (Binary data): Được lưu trữ vật lý trên ổ cứng (File System) của SERVER để tối ưu hiệu suất đọc/ghi.

DB. được lưu trữ ở bảng `users`, thuộc tính `avatar` (VARCHAR). frontend sẽ tự xử lý logic hiển thị dựa trên string này.

Chương 3

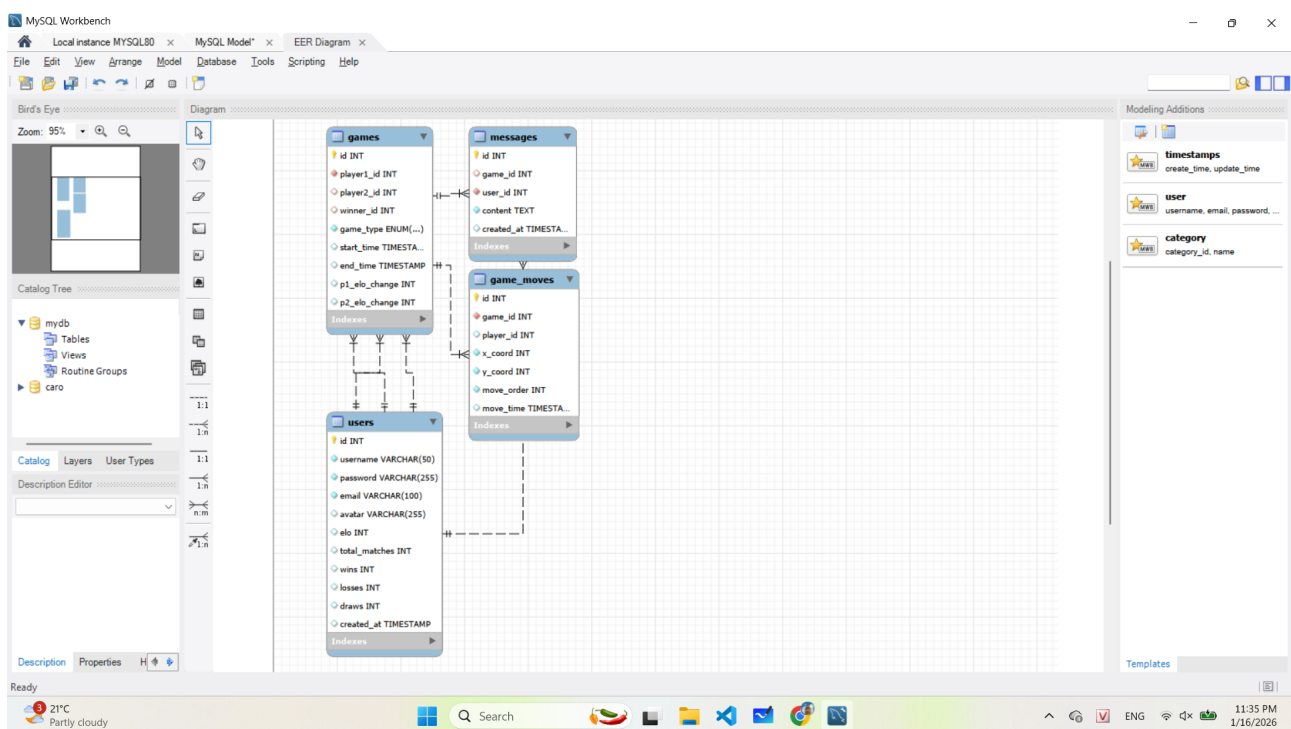
Thiết kế chương trình

3.1 Thiết kế kiến trúc

Hệ thống được xây dựng dựa trên mô hình kiến trúc Client-Server đa tầng, áp dụng triệt để mẫu thiết kế MVC (Model-View-Controller) để đảm bảo sự tách biệt rõ ràng giữa các thành phần xử lý (Separation of Concerns).

- **Model (Tầng Dữ liệu):** Sử dụng **MySQL** làm hệ quản trị cơ sở dữ liệu quan hệ, chịu trách nhiệm lưu trữ và toàn vẹn dữ liệu cho các thực thể cốt lõi như Người dùng (Users) và Lịch sử trận đấu (Games). Các truy vấn dữ liệu được đóng gói gọn gàng để tương tác trực tiếp với DB.
- **Controller (Tầng Điều khiển):** Được triển khai trên nền tảng **Node.js** và **Express.js**, đóng vai trò là bộ não xử lý logic nghiệp vụ trung tâm. Tại đây, các Controller (như `authController`, `gameController`) tiếp nhận yêu cầu từ Client, thực thi các thuật toán phức tạp (tính điểm Elo, xác thực JWT, thuật toán Minimax AI), sau đó điều phối dữ liệu trả về thông qua chuẩn **RESTful API** hoặc giao thức thời gian thực **WebSocket (Socket.IO)**.
- **View (Tầng Giao diện):** Được tách biệt hoàn toàn thành một ứng dụng **ReactJS (Single Page Application)**. Tầng này không chỉ đơn thuần hiển thị dữ liệu nhận được từ Controller mà còn quản lý trạng thái ứng dụng (State Management) và tương tác trực tiếp với người dùng, mang lại trải nghiệm mượt mà và linh hoạt.

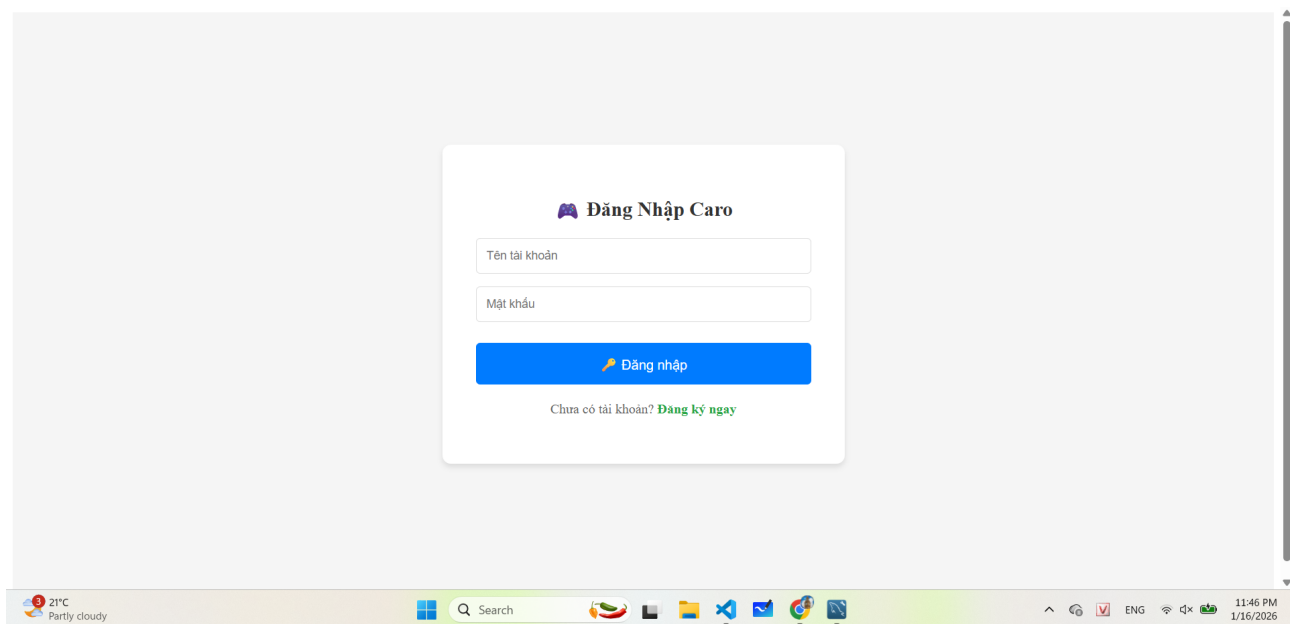
3.2 Thiết kế CSDL



[0] CSDL

Chương 4

Kết quả chương trình minh họa



[1] Register/Login

Chào mừng, long3!



[Đổi Avatar](#)

Elo hiện tại	Tỉ lệ thắng	Chi tiết kết quả
1035	61.5% Tổng: 13 ván	• Thắng: 8 (61.5%) • Thua: 5 (38.5%) • Hòa: 0 (0.0%)

[Chơi PvP \(Ghép cặp\)](#)
[Chơi PvE \(Với Máy\)](#)
[Chơi PvF \(Với Bạn bè\)](#)
[Xem Lịch Sử & Profile](#)
[Đăng xuất](#)

[2] Dashboard/Statistics/Logout

CHẾ ĐỘ: PVP

← Thoát về Dashboard

Đối thủ: long4

Bạn cảm quân: **X**

🔥 ĐẾN LƯỢT BẠN 🔥

● Trò chuyện

☒ Guri

[3] PvP

CHẾ ĐỘ: PVE

[← Thoát về Dashboard](#)

Đối thủ: **Mây (AI)**

Bạn cảm quân: **X**

🔥 ĐẾN LƯỢT BẠN 🔥

[4] PvE

CHẾ ĐỘ: PvF

← Thoát về Dashboard

Đối thủ: long4

Bạn cầm quân: X

Mã phòng: 16268

🔥 ĐẾN LƯỢT BẠN 🔥

Tro chuyện

Nhập tin nhắn...

Gửi

[5] PvF

← Quay lại Dashboard

Lịch Sử Ván Đấu

Tim theo tên đối thủ...

mm/dd/yyyy

Tim kiếm

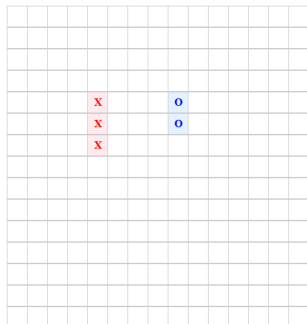
Ngày diễn ra	Đối thủ	Kết quả	Biến động Elo	Hành động
16/01/2026, 23:49:15	long4	Thua	-17	Xem lại
16/01/2026, 23:48:11	long4	Thua	-19	Xem lại
15/01/2026, 14:48:02	long4	Thắng	+14	Xem lại
15/01/2026, 14:20:56	long4	Thắng	+15	Xem lại
15/01/2026, 14:19:58	long4	Thắng	+17	Xem lại
15/01/2026, 13:51:42	long4	Thua	-16	Xem lại
15/01/2026, 13:49:07	long4	Thua	-18	Xem lại
15/01/2026, 13:46:53	long4	Thắng	+15	Xem lại
15/01/2026, 01:13:16	long4	Thua	0	Xem lại
15/01/2026, 01:01:32	long4	Thắng	0	Xem lại
15/01/2026, 00:53:20	long4	Thua	0	Xem lại
15/01/2026, 00:52:24	long4	Thắng	0	Xem lại
14/01/2026, 19:18:50	long4	Thắng	0	Xem lại

[6] History Management, Elo Fluctuation

[← Quay lại Lịch sử](#)

Xem lại ván đấu #20

[Trở lại](#) Nước đi: 5 / 9 [Tiếp theo](#)



[7] Replay

[← Quay lại Dashboard](#)

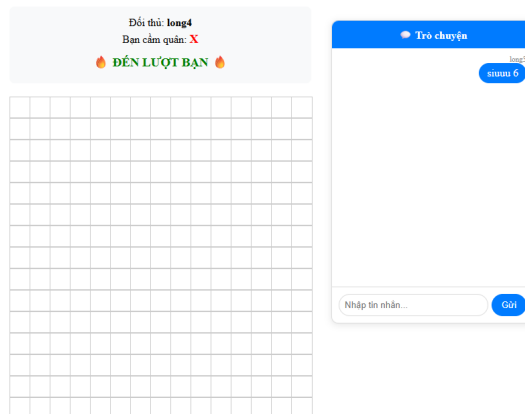
Lịch Sử Ván Đấu

Ngày diễn ra	Đối thủ	Kết quả	Biến động Elo	Hành động
14/01/2026, 19:18:50	long4	Thắng	0	Xem lại
14/01/2026, 19:11:41	long4	Thua	0	Xem lại

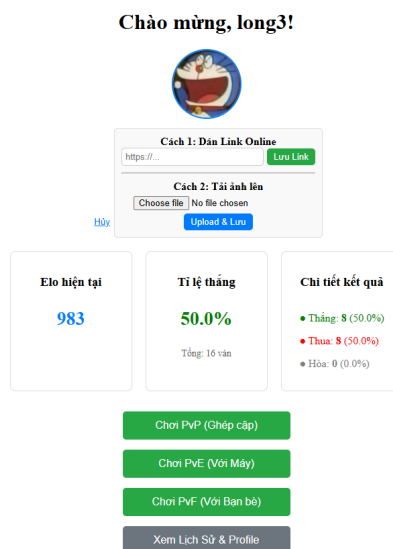
[8] Filter, Search

CHẾ ĐỘ: PVP

[← Thoát về Dashboard](#)



[9] Chat



[10] Profile Picture

Chương 5

Công cụ sử dụng

Frontend

Công nghệ	Logo	Loại (Type)	Mục đích sử dụng
HTML5		Language	Ngôn ngữ lập trình.
javascript ES6+		Language	Ngôn ngữ lập trình.
react.js(v18)		Library/Framework	Xây dựng giao diện người dùng theo kiến trúc Component (Single Page Application). Quản lý trạng thái (State) của bàn cờ và ứng dụng.
React Router DOM		Library (Routing)	Điều hướng trang mà không cần load lại (SPA). Quản lý các đường dẫn: /login, /register, /dashboard, /game, /history.
Socket.io (client)		Library (Real-time)	Kết nối hai chiều với Server. Nhận dữ liệu nước đi, tin nhắn chat và trạng thái phòng ngay lập tức (Real-time).
Axios		Library (HTTP Client)	Gửi các request HTTP (GET, POST, PUT) đến RESTful API của server để xử lý Đăng nhập, Đăng ký, Lấy lịch sử đấu, Upload ảnh.
Lucide-react		Library (Icons)	Library (Icons).
CSS3 & Inline Styles		Styling	Trang trí giao diện, bố cục bàn cờ (Grid layout), hiệu ứng màu sắc cho quân X/O.

Backend

Công nghệ	Logo	Loại (Type)	Mục đích sử dụng
Node.js		Runtime Environment	Môi trường chạy mã JavaScript phía server, xử lý các request bất đồng bộ.
Express.js		Framework	Xây dựng RESTful API, định tuyến (Routing) cho các request, cấu hình Middleware.
Socket.io (server)		Library (Real-time)	Tạo WebSocket server. Quản lý các phòng chơi (Rooms), ghép cặp người chơi (Matchmaking), và phát sóng (Broadcast) nước đi tới các client.
MySQL2		Database Driver	Thư viện giúp Node.js kết nối và thực thi câu lệnh SQL với cơ sở dữ liệu MySQL (sử dụng Connection Pool để tối ưu).
Bcryptjs		Security Library	Mã hóa (Hash) mật khẩu người dùng trước khi lưu vào database và kiểm tra mật khẩu khi đăng nhập (tăng tính bảo mật).
Jsonwebtoken (JWT)		Security Library	Tạo và xác thực Token đăng nhập. Giúp duy trì phiên làm việc (Session) theo cơ chế Stateless (không lưu session trên server).
Multer		Middleware	Xử lý <code>multipart/form-data</code> để hỗ trợ tính năng Upload Avatar (lưu file ảnh từ Client lên ổ cứng Server).
Cors		Middleware	Cho phép trình duyệt (chạy ở port 3000) gọi API sang Server (chạy ở port 5000) mà không bị chặn bởi chính sách bảo mật (Cross-Origin Resource Sharing).
Dotenv		Library (Config)	Quản lý các biến môi trường (như DB_PASS, JWT_SECRET) để bảo mật thông tin nhạy cảm, không fix cứng trong code.

Backend

Công nghệ	Logo	Loại (Type)	Mục đích sử dụng
MySQL		RDBMS	Hệ quản trị cơ sở dữ liệu quan hệ. .

Chương 6

Kết luận, hướng phát triển

6.1 Kết luận

6.2 Hướng phát triển

Về định hướng mở rộng trong tương lai, em dự kiến sẽ nâng cấp hệ thống theo 3 hướng chính.

- Thứ nhất là tối ưu hóa thuật toán AI, áp dụng thêm Machine Learning để máy có thể tự học qua các ván đấu thay vì chỉ dựa trên thuật toán Minimax cố định.
- Thứ hai là triển khai (Deployment) dự án lên các nền tảng đám mây thực tế (như AWS hoặc Vercel/Render) và tích hợp CI/CD để tự động hóa quy trình cập nhật.
- Cuối cùng, em sẽ phát triển phiên bản Mobile App (sử dụng React Native) dùng chung Backend hiện có, đồng thời bổ sung các tính năng mạng xã hội như danh sách bạn bè, chế độ khán giả (Spectator Mode) và bảng xếp hạng (Leaderboard) toàn server để tăng tính cạnh tranh và gắn kết cho cộng đồng user.

Chương 7

Tài liệu tham khảo